



Nädal 8

Nadal 8: Python API'd ja Automatiseerimine

Supabase API + Python Integratsioon — Marko Saare Valjakutse

Marko Saar, Product Manager

Sessiooni ülevaade (90 min)

Faas	Aeg	Sisu
CONNECTION	0:00–0:15	Marko narratiiv: VIP-nimekiri on vananenud, API motivatsioon
CONCEPTS	0:15–0:45	REST API pohitoed, Supabase Python client demo
CONCRETE PRACTICE	0:45–1:20	3 harjutust: API fetch, filtreerimine, veakasitlus
CONCLUSIONS	1:20–1:30	Kokkuvote, KKK, Sessioon 2 eelvaade



CONNECTION

0:00–0:15

Marko narratiiv ja API motivatsioon

Marko Saare sonum

Marko Saar, Product Manager

"Hey, DataDriven team! Last week oli AMAZING — ma sain mu VIP listed! Aga tana... need on juba vananenud. Uued tellimused tulid sisse. Ma tahan, et see skript update'ib AUTOMAATSELT iga nadal. Kas see on võimalik?"



Toomas Kaski kommentaar

Toomas Kask, IT Director

"Marko on õigesti marganud — PRODUCTION analytics ei ole uhekordne skript. See on SUSTEEM. Automatiseeritud, testitud, monitored."



Nadal 7 vs Nadal 8 — evolutsioon

Nadal 7 (staatiline)

- ◆ CSV fail -> pandas -> RFM
- ◆ Uhekordne skript
- ◆ Kasimoodus: laadi fail, jookse kood
- ◆ Tulemused kiiresti vananevad

Nadal 8 (automaatne)

- ◆ Supabase API -> pandas -> RFM -> CSV
- ◆ Automatiseeritud pipeline
- ◆ Automaatne: API parib varikaid andmeid
- ◆ Tulemused alati varikised



CONCEPTS

0:15–0:45

REST API pohitoid ja Supabase Python client

Mis on API?

REST API lihtsalt seletatuna

- ◆ API on nagu restoranis kelner — sa ei lae kokki, sa tellid kelnerilt
- ◆ API on programmide vaheline kelner: sinu kood raaagib andmebaasiga
- ◆ REST = Representational State Transfer (standardne veebiprotokoll)
- ◆ Supabase annab automaatselt REST API iga tabeli jaoks



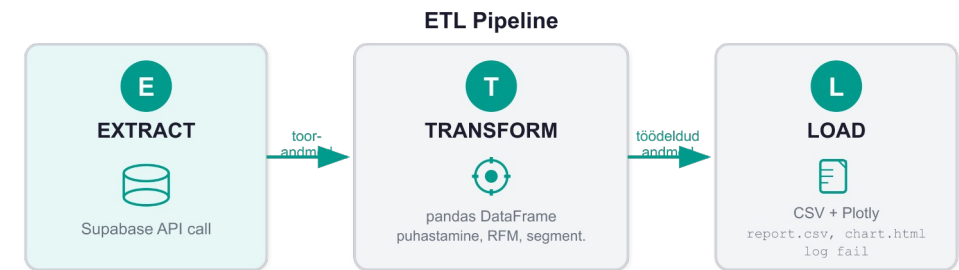
HTTP meetodid — SQL ekvivalendid

HTTP meetod	Tegevus	SQL ekvivalent
GET	Loe andmeid	SELECT
POST	Loo uusi andmeid	INSERT
PUT	Uuenda andmeid	UPDATE
DELETE	Kustuta andmeid	DELETE



ETL Pipeline arhitektuur

- ◆ EXTRACT — andmete parimine Supabase API kaudu
- ◆ TRANSFORM — andmete tootlemine pandas abil (RFM, puhastamine)
- ◆ LOAD — tulemuste salvestamine (CSV, dashboard, raport)
- ◆ Iga etapp on eraldi funktsioon — kerge testida ja debugida



Supabase Python client — seadistamine

```
# 1. Installimine
# pip install supabase-py python-dotenv

# 2. .env faili loomine (MITTE KUNAGI COMMITI!)
# SUPABASE_URL=https://xxx.supabase.co
# SUPABASE_KEY=eyJhbGciOi...
```

```
import os
from dotenv import load_dotenv
from supabase import create_client

load_dotenv() # Laeb .env failist

url = os.getenv('SUPABASE_URL')
key = os.getenv('SUPABASE_KEY')
supabase = create_client(url, key)
print('Supabase uhendus loodud!')
```

.env fail hoia saladusi turvaliselt. Lisa .env kohe .gitignore faili! Mitte kunagi commiti API voti Giti!

Esimehe API paring — andmed DataFrame'iks

```
import pandas as pd

# Pari esimesed 5 rida
response = supabase.table('sales').select('*').limit(5).execute()
df = pd.DataFrame(response.data)
print(f'Fetched {len(df)} rows from Supabase API')
print(df.head())

# response.data on list of dicts
# pandas teisendab selle automaatselt DataFrame'iks
```

response.data on Python list, kus iga element on dictionary. pd.DataFrame() teeb sellest tuttava tabeli.

CONCRETE PRACTICE

0:45–1:20

3 harjutust: API fetch, filtreerimine, veakasitus

Harjutus 1: Asenda CSV API'ga

Asendage oma Nadala 7 CSV laadimine Supabase API paringuga. Kontrollige, et saate sama arvu ridu.

```
# OLD WAY (Week 7)
df = pd.read_csv('urbanstyle_sales.csv')

# NEW WAY (Week 8)
response = supabase.table('sales').select('*').execute()
df = pd.DataFrame(response.data)
print(f'Fetched {len(df)} rows from Supabase API')
```

💡 Vihje: Kui saate veateate, kontrollige .env faili ja API voti!



Harjutus 2: Filtreeritud paringud

Kasutage Supabase Python client'i filtrimetodeid, et parida ainult vajalikud andmed.

```
# Tallinna muugid
response = supabase.table('sales') \
    .select('*') \
    .eq('city', 'Tallinn') \
    .execute()
print(f'Tallinna muuke: {len(response.data)}')

# Viimase 30 päeva muugid
response = supabase.table('sales') \
    .select('*') \
    .gte('order_date', '2025-02-01') \
    .execute()
print(f'Viimase kuu muuke: {len(response.data)}')
```

💡 Vihje: `.eq()` = equals, `.gte()` = greater than or equal, `.lt()` = less than



Harjutus 3: Veakasitlus

Lisage oma koodile try/except plokk. Tootmiskoodis ei tohi viga programmi kokku jooksutada!

```
try:
    response = supabase.table('sales') \
        .select('*').execute()
    df = pd.DataFrame(response.data)
    print(f'Success: {len(df)} rows')
except Exception as e:
    print(f'ERROR: {e}')
    print('Kontrolli .env faili ja API votit!')
    # Fallback: kasuta CSV faili
    df = pd.read_csv('datasets/clean/sales.csv')
    print(f'Fallback CSV: {len(df)} rows')
```

 *Vihje: Alati planeeri fallback-lahendus! Mis juhtub, kui API on maas?*



CONCLUSIONS

1:20–1:30

Kokkuvote ja jargmised sammud

Kokkuvote — mida me tana oppisime?

- ◆ REST API pohitoed: GET/POST/PUT/DELETE = SQL ekvivalendid
- ◆ Supabase Python client: `create_client()`, `.table().select().execute()`
- ◆ Filtreerimine API tasemel: `.eq()`, `.gte()`, `.lt()` — vaidab vorguliiust!
- ◆ `.env` failide turvalisus — MITTE KUNAGI commiti saladusi Giti
- ◆ `response.data` -> `pd.DataFrame()` — tuttav toovooog jaab samaks



Kokkuvote — mida me tana oppisime? (jätkub)

- ◆ Veakasitus: try/except + fallback CSV



Jargmised sammud

Session 2 eelvaade ja kodutoo

- ◆ Session 2: ehitage meeskonnaga TERVE automatiseeritud pipeline (ETL)
- ◆ 4 rolli: A = Extract, B = Transform, C = Automate, D = Monitor
- ◆ Eesmark: `python pipeline.py --date 2025-03-01` peab tootama!
- ◆ Portfoolio: pipeline kood (.py) + README -> GitHub
- ◆ Marko tahab tulemusi, mitte protsessi — teie pipeline annab talle tulemusi AUTOMAATSELT



Jargmised sammud (jätkub)

- ◆ Sessioon 3 demo: relay-formaat — ajalimiit range, 45 sek per inimene

